A
Project Report
On
"DocAssist – AI-Powered Documentation Assistant with RAG Architecture"

Prepared by

Shruti Panchal (22CS044)

Under the guidance of

Prof. Abhishek Patel

A Report Submitted to

Charotar University of Science and Technology

for Partial Fulfillment of the Requirements for the

Degree of Bachelor of Technology

in Computer Science and Engineering

(8th Semester CSE403-MAJOR PROJECT)

Submitted at

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Chandubhai S. Patel Institute of Technology

At: Changa, Dist: Anand – 388421

April 2026

A
Project Report
On
"DocAssist – AI-Powered Documentation Assistant with RAG Architecture"

Prepared by

Shruti Panchal (22CS044)

Under the guidance of

Prof. Abhishek Patel

A Report Submitted to

Charotar University of Science and Technology

for Partial Fulfillment of the Requirements for the

Degree of Bachelor of Technology

in Computer Science and Engineering

(8th Semester CSE403-MAJOR PROJECT)

Submitted at

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Chandubhai S. Patel Institute of Technology

At: Changa, Dist: Anand – 388421

April 2026

CANDIDATE'S DECLARATION

I/We hereby declare that the project entitled **"DocAssist – AI-Powered Documentation Assistant with RAG Architecture"** is my/our own work conducted under the guidance of **Prof. Abhishek Patel** and **Mr. Kush Patel.**

I further declare that to the best of my knowledge, the project for B. Tech does not contain any part of the work, which has been submitted for the award of any degree either in this University or in other University without proper citation.

Shruti Panchal
(22CS044)

Prof. Abhishek Patel
Assistant Professor,
Department of Computer Science and Engineering,
Faculty of Technology & Engineering,
Changa – 388425.

Trionic

Date : April, 2026

TO WHOM IT MAY CONCERN

This is to certify that **Shruti Panchal**, a student of Charotar University of Science and Technology (CSPIT), is currently undergoing an internship with Trionic Technologies LLP as a AI/ML Intern.

She joined the organization on January 1, 2026, and her internship is ongoing for a duration of 6 months.

During this period, she has been involved in working on research and development activities related to Retrieval-Augmented Generation (RAG), including data processing, information retrieval, and AI-based system development.

Her performance has been satisfactory, and she has shown a positive attitude towards learning and assigned responsibilities. This certificate is issued for academic purposes to confirm that her internship is currently ongoing.

Yours sincerely,

Kush Patel,
Co-founder & CEO,
Trionic Technologies LLP

+91 7984222871
hello@trionic.co.in

CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

CERTIFICATE

This is to certify that the report entitled "**DocAssist – AI-Powered Documentation Assistant with RAG Architecture**" is a bonafied work carried out by **SHRUTI PANCHAL (22CS044)** under the guidance and supervision of **Prof. Abhishek Patel & Mr. Kush Patel** for the subject **CSE403-MAJOR PROJECT** of 8th Semester of Bachelor of Technology in **Computer Science and Engineering** at Faculty of Technology & Engineering – CHARUSAT, Gujarat.

To the best of my knowledge and belief, this work embodies the work of candidate herself, has duly been completed, and fulfills the requirement of the ordinance relating to the B.Tech. Degree of the University and is up to the standard in respect of content, presentation and language for being referred to the examiner.

Under supervision of,

Prof. Abhishek Patel
Assistant Professor
Department of Computer Science and Engineering
CSPIT, Changa, Gujarat.

Mr. Kush Patel
Co-founder & CEO
Trionic Technologies LLP.

Dr. Amit Thakkar
Head & Professor
Department of Computer Science and Engineering
CSPIT, Changa, Gujarat.

ABSTRACT

In this report, I present my work on the development of **DocAssist**, an AI-powered assistant based on a **Retrieval-Augmented Generation (RAG)** architecture. The project focuses on improving the reliability of **Large Language Models (LLMs)** by reducing hallucinations and enabling responses grounded in **domain-specific** documentation.

The main objectives were to build a document ingestion and **semantic retrieval** pipeline and integrate it into an interactive system capable of generating accurate, **context-aware responses** with source attribution. The system was developed using a full-stack architecture with Next.js, Node.js, a Python-based AI pipeline, and MongoDB Atlas for vector search.

The project was carried out in phases, starting with data preprocessing and indexing, followed by the development of **retrieval**, **response generation**, and **a streaming chat interface**. This report summarizes the design, implementation, and key improvements made to deliver a reliable and efficient assistant for developer support.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my academic mentor, **Abhishek Patel**, for his continuous guidance and support throughout the project. His insights and direction were invaluable in shaping the academic structure of this work.

I am also deeply thankful to my industry mentor, **Kush Patel**, for his technical guidance and valuable suggestions, especially in understanding RAG architectures and system design, which played a crucial role in the successful development of DocAssist.

I would like to extend my gratitude to the Head of Department, **Dr. Amit Thakkar**, and all faculty members of the **Chandubhai S. Patel Institute of Technology** for their support, encouragement, and for providing the necessary resources to carry out this project.

Finally, I would like to thank my peers and colleagues for their cooperation and support during the development and documentation of this project. Their ideas and feedback helped improve the overall quality of the work.

Shruti Panchal (22CS044)

LIST OF FIGURES

LIST OF TABLES

TABLE OF CONTENTS

1. INTRODUCTION

1.1 PROJECT OVERVIEW

Modern Large Language Models (LLMs) have improved information access but often suffer from issues like hallucinations and lack of grounding in **domain-specific documentation**, leading to unreliable responses in technical scenarios.

For developers, retrieving accurate information from large documentation sources is time-consuming, and traditional keyword-based search systems often fail to capture context and intent. This creates a need for more intelligent, context-aware solutions.

To address this, the project proposes **DocAssist**, an AI-powered assistant based on a Retrieval-Augmented Generation (RAG) architecture. The system combines document retrieval with generative capabilities to deliver accurate, context-aware responses supported by relevant citations.

The approach aims to improve the efficiency, reliability, and transparency of developer support systems by providing precise and trustworthy answers grounded in documentation.

1.2 OBJECTIVE

The primary objectives of this project are:

- To design and implement a scalable Retrieval-Augmented Generation (RAG) pipeline, including data ingestion, chunking, embedding, and retrieval.
- To develop a modular backend system supporting authentication, conversation management, and seamless integration between services.
- To build an interactive frontend with real-time response streaming for enhanced user experience.
- To enable accurate and context-aware responses by grounding outputs strictly in retrieved documentation.
- To create a complete end-to-end system, including documentation scraping, vector indexing, and a full-stack chat interface.

1.3 SCOPE

The scope of this project includes the end-to-end development of the **DocAssist** platform, focusing on building a reliable and scalable AI-powered assistant for developer support. It covers the automated scraping of technical documentation (such as Stripe and Next.js), followed by preprocessing, chunking, and embedding to create a structured knowledge base. The project also includes implementing a vector-based retrieval system using MongoDB Atlas to enable efficient semantic search.

Additionally, the scope involves the development of a full-stack application, including a modular backend for handling APIs, authentication, and conversation management, along with a user-friendly frontend that supports real-time response streaming. The system is designed to generate accurate, context-aware responses with proper source attribution.

1.4 TOOLS AND TECHNOLOGIES USED

The development of the DocAssist system involved a combination of frontend, backend, database, and AI-related technologies to build a scalable and efficient RAG-based application.

1.4.1 Frontend Technologies

Technology	**Purpose in System**
Next.js (React Framework)	Used to develop the user interface and manage routing
TypeScript	Ensures type safety and improves code maintainability
Tailwind CSS	Used to design responsive and modern UI components

Table 1.4.1 Frontend technologies

1.4.2 Backend Technologies

Technology	**Purpose in System**
Node.js with Express.js	Handles API requests, business logic, and service integration
TypeScript	Enables scalable and structured backend implementation
Server-Sent Events (SSE)	Used to stream responses from backend to frontend in real time

Table 1.4.2 Backend Technologies

1.4.3 AI and Embedding service

Technology	Purpose in System
Python (Flask)	Used to build embedding service API
Sentence-Transformers (Hugging Face)	Converts text into vector representations
Retrieval-Augmented Generation (RAG)	Retrieves relevant context and generates accurate responses

Table 1.4.3 AI and Embedding services

1.4.4 Database

Technology	Purpose in System
MongoDB Atlas	Stores document chunks and embeddings
MongoDB Vector Search	Retrieves relevant content using vector similarity

Table 1.4.4 Database

1.4.5 Integration and APIs

Technology	Purpose in System
REST API	Enables interaction between backend and embedding service
Groq API (LLM)	Generates responses based on retrieved context

Table 1.4.5 Integration and APIs

2. PROJECT MANAGEMENT

2.1 PROJECT PLANNING

The development of the DocAssist system was carried out using an **incremental and iterative approach**. The project was divided into multiple phases, starting from understanding the problem statement and studying relevant technologies, followed by system design, implementation of core modules, integration of components, and final testing.

Initially, the focus was on identifying the requirements of a documentation-based assistant and selecting appropriate technologies such as Next.js, Node.js, MongoDB Atlas, and embedding models. A structured plan was then created to implement the system in stages, ensuring that each component was developed and tested before integration.

2.1.1 Project Development Approach and Justification

An **incremental development approach** was adopted for this project. The system was developed in multiple stages, where each stage added new functionality to the existing system.

The project began with the development of a basic chat interface and backend structure. This was followed by the implementation of the RAG pipeline, including data scraping, chunking, embedding generation, and vector storage. Later stages focused on integrating multiple documentation sources, enabling context-specific retrieval, and adding advanced features such as citation support.

This approach was chosen because:

- It allowed gradual integration of complex components like RAG and vector search
- It enabled continuous testing and debugging at each stage
- It reduced the risk of system failure by validating each module independently
- It provided flexibility to incorporate improvements based on intermediate results

2.2 PROJECT WORK SCHEDULING

The project work was systematically scheduled over a period of five months, with each month focusing on specific objectives and deliverables. The schedule included phases such as requirement analysis, system design, implementation of core functionalities, integration of components, and system enhancement.

A Gantt chart was used to visually represent the timeline and distribution of tasks throughout the project duration. It helped in tracking progress, ensuring timely completion of milestones, and maintaining a structured workflow.

The major phases of the project included:

- Requirement analysis and technology selection
- System architecture design
- Implementation of RAG pipeline
- Multi-document integration and context isolation
- Testing, debugging, and feature enhancement (including citations)

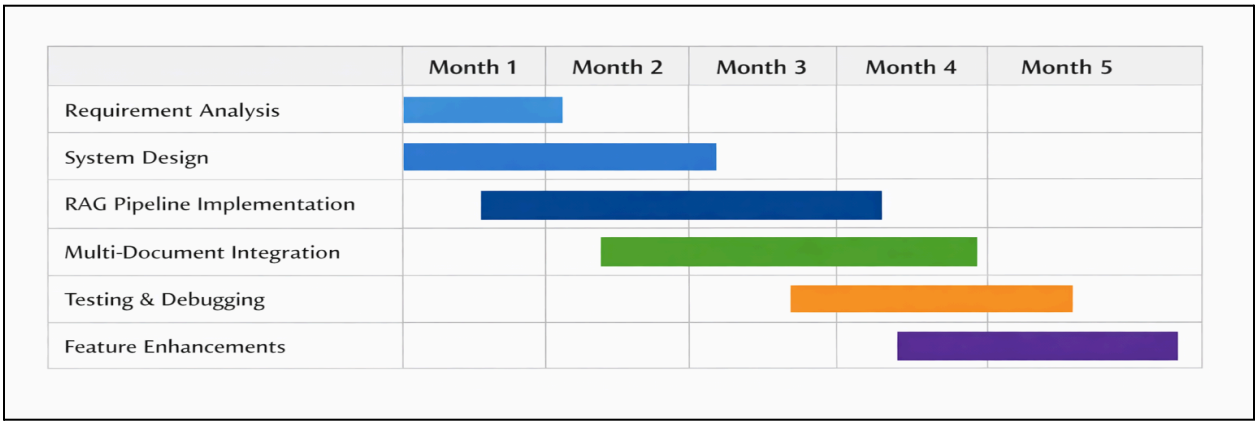

Figure 2.1 Gantt Chart for Project Scheduling

3. SYSTEM REQUIREMENT STUDY

3.1 USER CHARACTERISTICS

The DocAssist system is designed to serve users who interact with technical documentation through a conversational interface. The system primarily targets developers, students, and technical users who need quick and accurate information from documentation sources such as LiveKit, Stripe, and Next.js.The users are expected to have basic familiarity with web applications and technical documentation. Different user roles interact with the system in distinct ways, as described below.

3.1.1 End Users(Developers/Students)

- Users who query documentation using natural language
- May have varying levels of technical knowledge
- Expect accurate, context-specific responses
- Interact with the chat interface to retrieve information

3.1.2 System Administrator

- Manages system configuration and data ingestion
- Responsible for adding new documentation sources
- Monitors system performance and database

3.1.3 AI System (RAG Engine)

- Processes user queries and generates embeddings
- Retrieves relevant document chunks using vector search
- Generates responses using LLM based on retrieved context

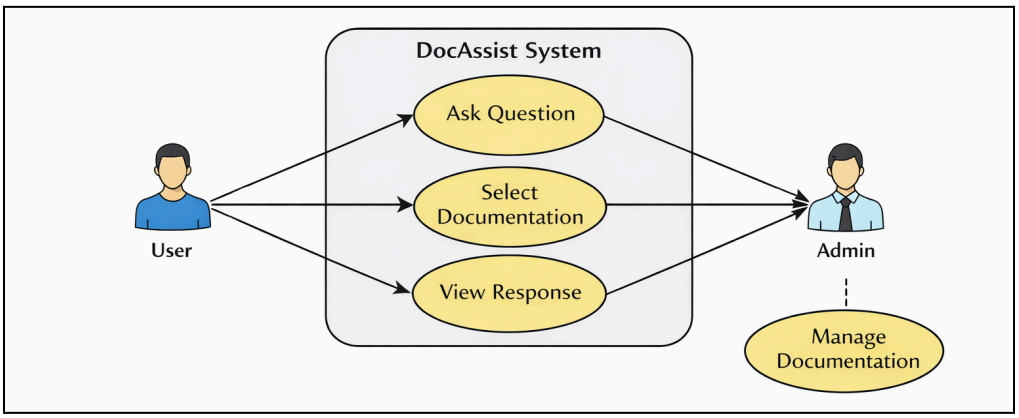

Figure 3.1 Use Case Diagram of Docassist system

3.2 HARDWARE & SOFTWARE REQUIREMENTS

The successful deployment and functioning of the DocAssist system requires specific hardware and software configurations. The following section outlines the minimum requirements necessary to ensure smooth operation of the system.

3.2.1 Hardware Requirements

Component	**Requirement**
Processor	Minimum Intel i5 or equivalent
RAM	Minimum 8 GB
Storage	Minimum 10 GB free space
Internet	Required for API communication

Table 3.2.1 Hardware requirements

3.2.2 Software Requirements

Software	**Purpose**
Node.js	Backend server
Next.js	Frontend Framework
Python (Flask)	Embedding service
MongoDB Atlas	Database & vector storage
VS Code	Development environment

Table 3.2.2 Software requirements

3.2 ASSUMPTIONS AND CONSTRAINTS

This section outlines the key assumptions made during the development of the DocAssist system and the constraints that may impact its performance and functionality. These factors help define the operational boundaries of the system.

3.2.1 Assumptions

- Users have stable internet connectivity
- Documentation websites are publicly accessible
- Embedding service remains available during query processing
- Users provide meaningful queries related to documentation

3.2.2 Constraints

- System depends on third-party APIs (LLM, embeddings)
- Performance may vary based on dataset size
- Limited to predefined documentation sources
- Real-time response depends on network latency

4. SYSTEM ANALYSIS

4.1 STUDY OF EXISTING SOLUTION

Currently, users rely on traditional methods such as manually browsing documentation websites (e.g., Stripe, Next.js, LiveKit) or using keyword-based search to find relevant information. These approaches require users to navigate through multiple pages, which can be time-consuming and inefficient.

Some platforms provide search functionality, but they often lack contextual understanding and fail to deliver precise answers to user queries. Additionally, users must interpret and extract relevant information manually from lengthy documentation pages.

4.1.1 Key Features of Existing Documentation Systems

- Keyword-based search functionality
- Structured documentation with categorized sections
- Static content presentation (no conversational interface)
- Navigation through menus and hyperlinks
- Limited or no personalization based on user queries

4.2 LIMITATION OF EXISTING SOLUTION

- Lack of conversational interaction for querying documentation
- Time-consuming process to locate relevant information
- No context-aware understanding of user queries
- Requires manual reading and interpretation of content
- No integration of multiple documentation sources in a single interface
- Absence of intelligent retrieval mechanisms like vector search

4.3 REQUIREMENT OF PROPOSED SOLUTION

The proposed system, DocAssist, aims to overcome the limitations of traditional documentation systems by providing an AI-powered conversational interface. It enables users to ask questions in natural language and receive accurate, context-based answers.

The system integrates multiple documentation sources and uses a Retrieval-Augmented Generation (RAG) approach to retrieve relevant content and generate responses. It also ensures that answers are restricted to the selected documentation, improving accuracy and reliability.

4.3.1 Functional Requirements

- User authentication and session management
- Ability to select documentation (e.g., LiveKit, Stripe, Next.js)
- Accept user queries in natural language
- Generate embeddings for user queries
- Retrieve relevant document chunks using vector search
- Generate responses using LLM based on retrieved context
- Stream responses in real-time using SSE
- Display citations (source title and URL) along with responses
- Maintain conversation history

4.3.2 Non-Functional Requirements

- **Performance:** System should provide responses with minimal latency
- **Scalability:** Should support multiple users and documentation sources
- **Reliability:** System should handle failures in API or services gracefully
- **Usability:** Interface should be simple and user-friendly
- **Security:** Ensure secure authentication and data handling
- **Maintainability:** Codebase should be modular and easy to update

4.4 SYSTEM WORKFLOW

The workflow of the DocAssist system follows a structured pipeline that integrates frontend interaction,backend processing, and AI-based retrieval:

1. User selects a documentation source from the interface
2. User enters a query in natural language
3. Backend sends query to Python embedding service
4. Embedding is generated using sentence-transformers
5. MongoDB Atlas performs vector search to retrieve relevant chunks
6. Retrieved context is passed to the LLM (Groq API)
7. LLM generates a response based only on the provided context
8. Response is streamed to the frontend using SSE
9. Citations are displayed along with the response

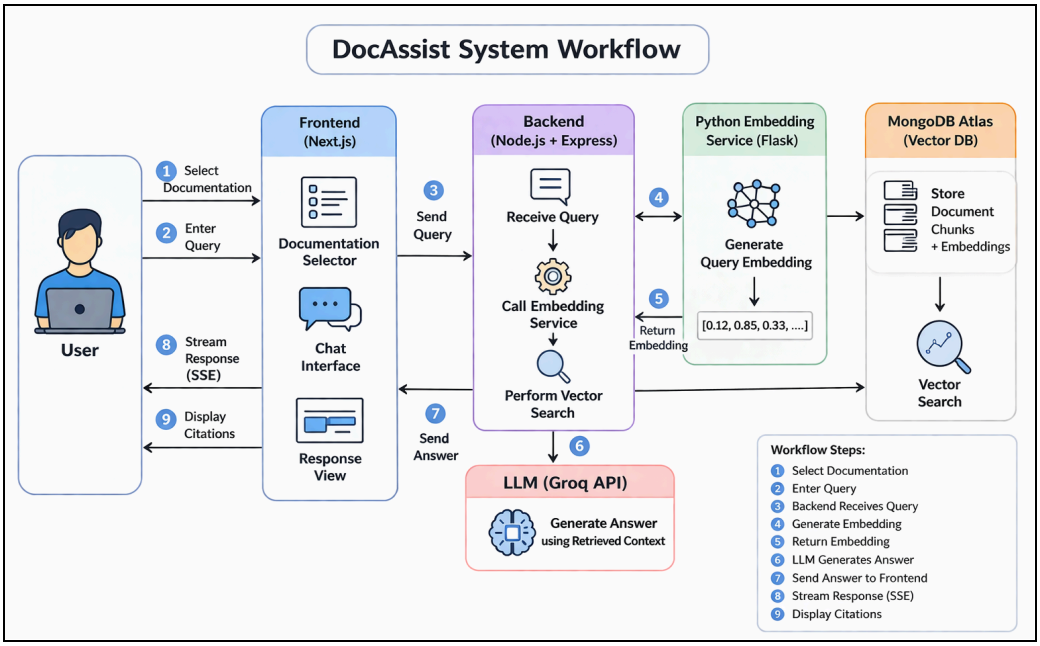

Figure 4.4.1 System Workflow of Docassist

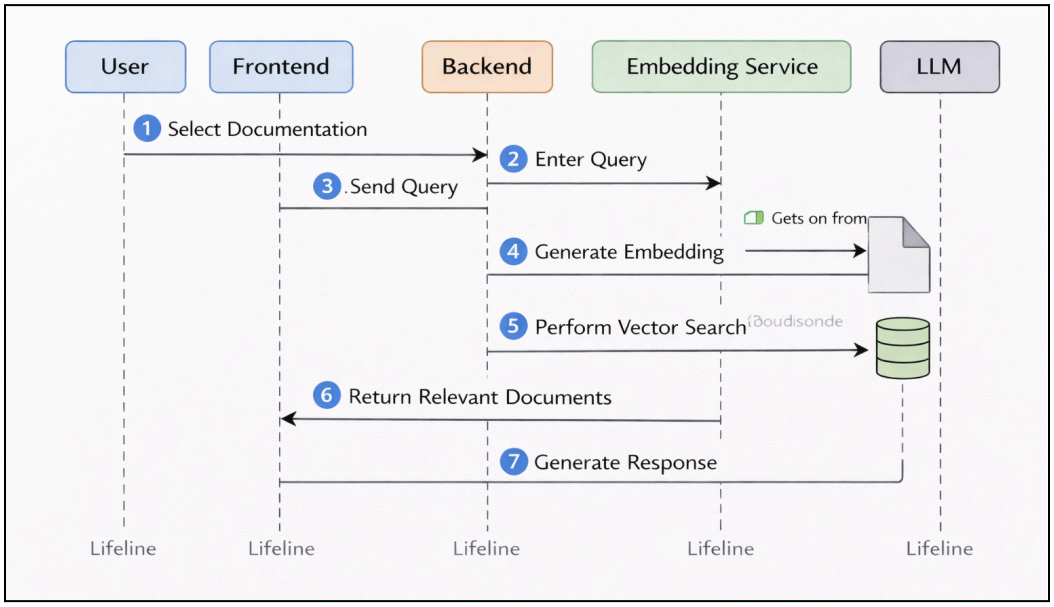

Figure 4.4.2 Sequence Diagram of Docassist

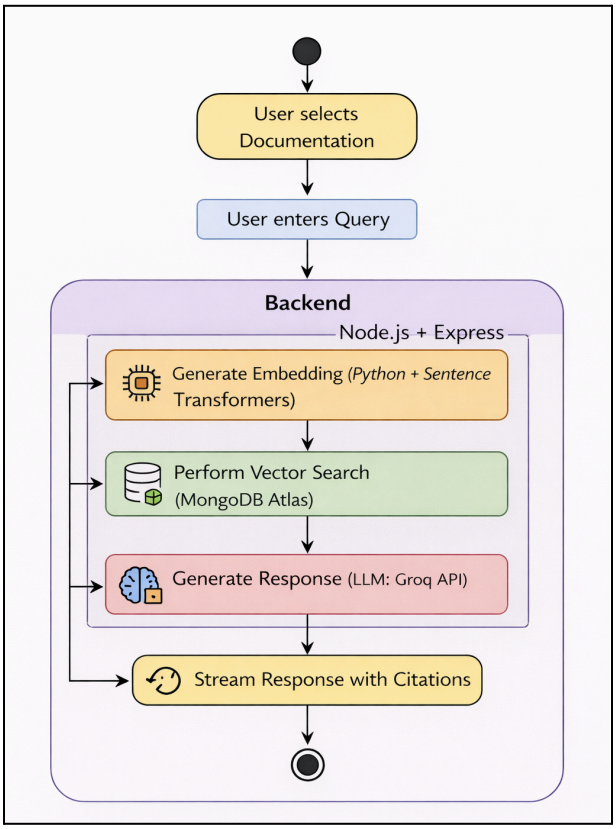

Figure 4.4.3 Activity Diagram of Docassist

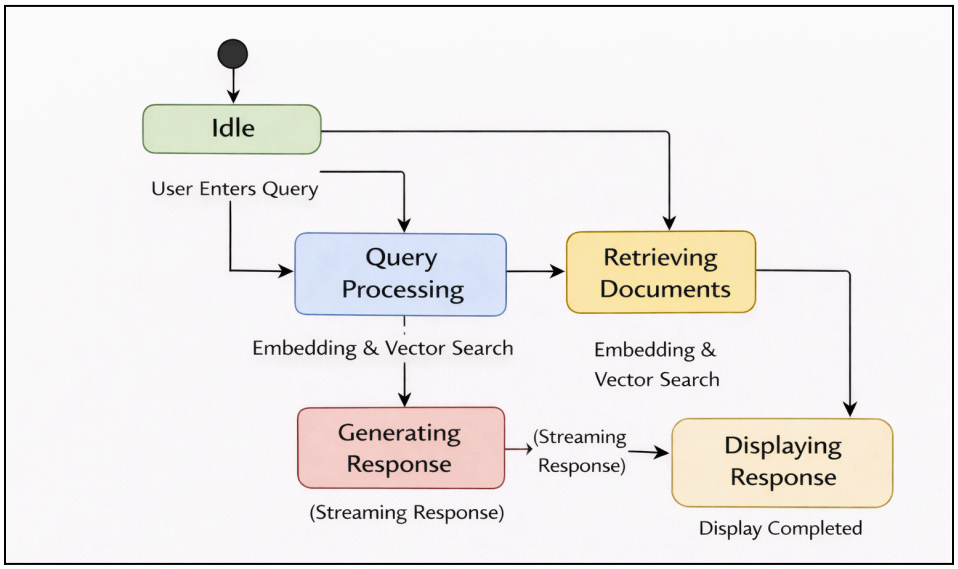

Figure 4.4.4 State Diagram of Docassist

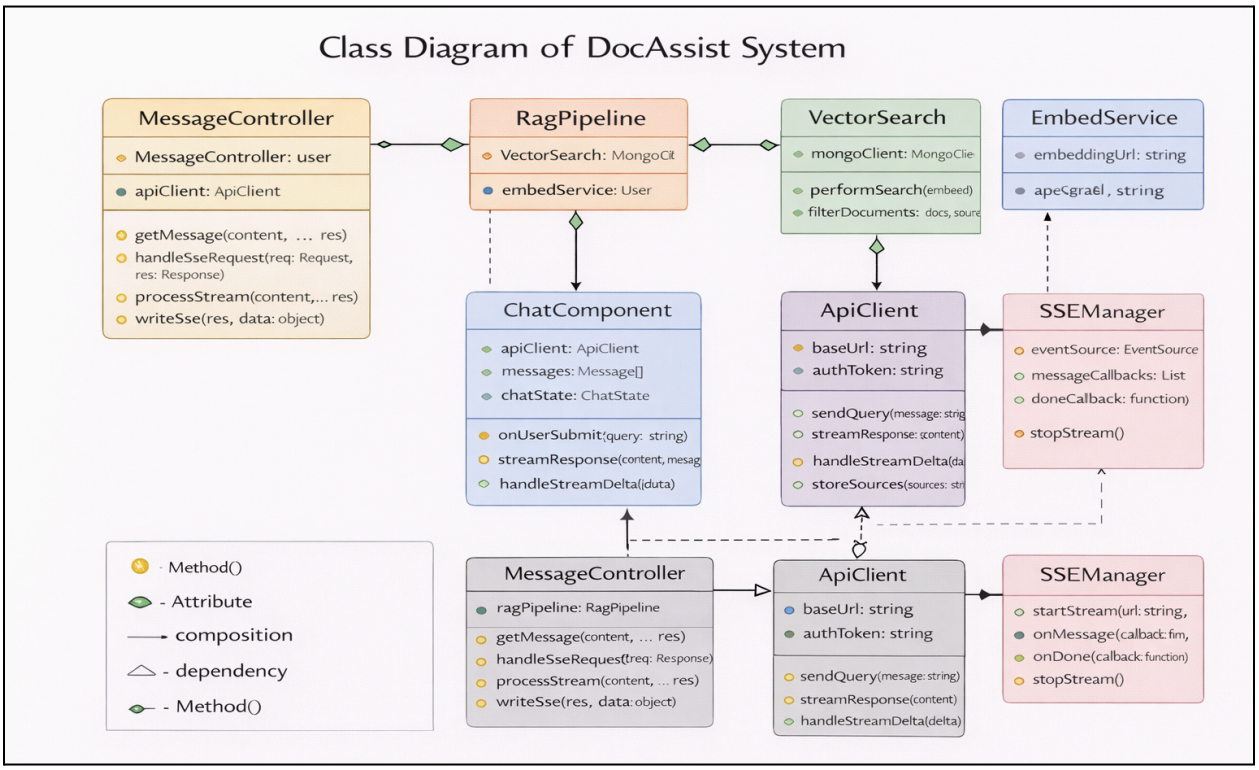

Figure 4.4.5 Class Diagram of Docassist

5. SYSTEM DESIGN

5.1 SCREEN LAYOUT

The DocAssist system provides a clean and user-friendly interface that allows users to interact with technical documentation through a chat-based system. The interface is designed to ensure ease of use, quick navigation, and efficient interaction with the system.

The main screen consists of a documentation selector, chat interface, and response display area. Users can select a specific documentation source and ask queries in natural language, receiving responses in real time.

5.1.1 Frontend Interface (Chat UI)

The frontend of the system is developed using Next.js and provides the following components:

- **Documentation Selector:** Allows users to choose between multiple documentation sources (LiveKit, Stripe, Next.js, etc.)
- **Chat Input Box:** Enables users to enter queries in natural language
- **Chat Display Area:** Shows conversation history between user and system
- **Response Section:** Displays generated answers along with citations
- **Streaming Response:** Responses are streamed in real-time using SSE

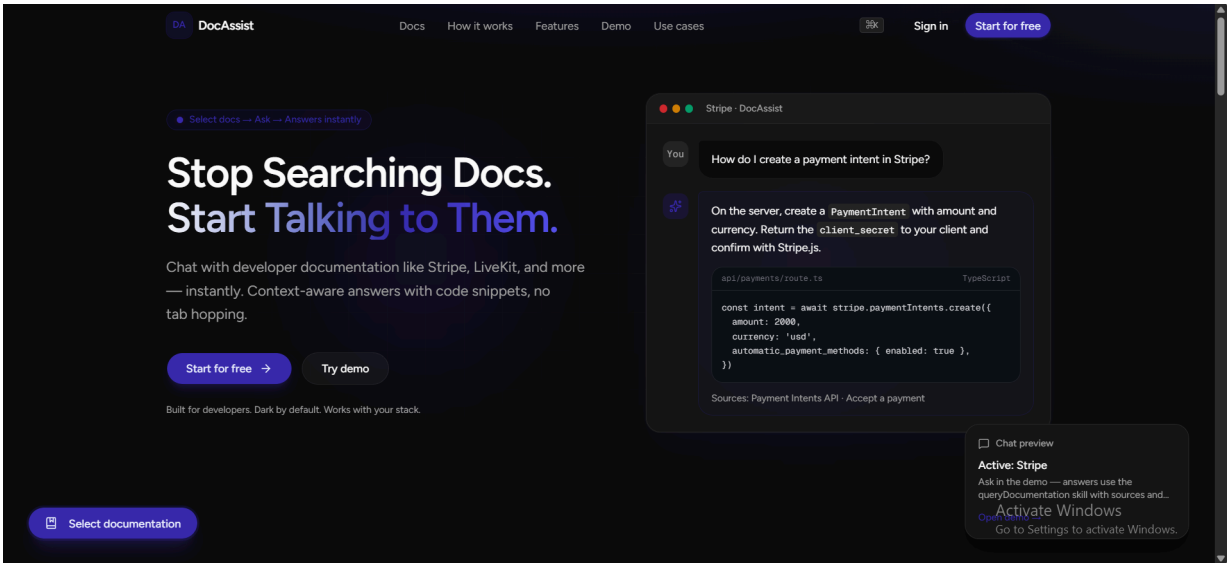

Figure 5.1.1 Home Page of Docassist

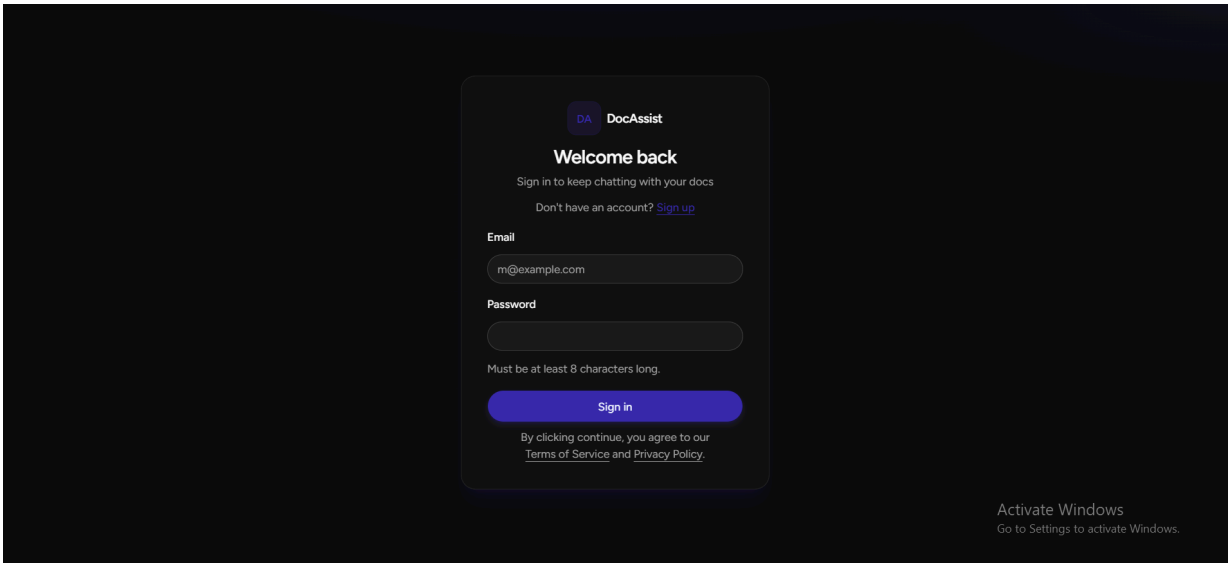

Figure 5.1.2 Login Page of Docassist

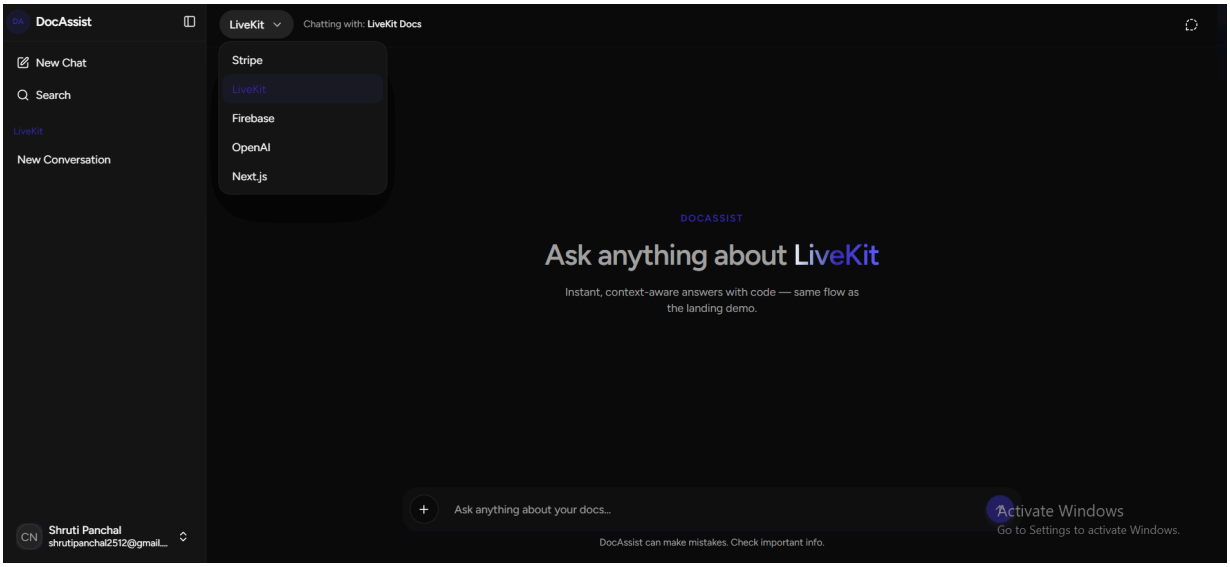

Figure 5.1.3 Chat Session of Docassist

5.2 METHOD PSEUDO CODE

This section describes the core methods used in the DocAssist system for **processing user queries**, **retrieving** relevant information, and **generating** responses.

5.2.1 Query Processing and Embedding Generation

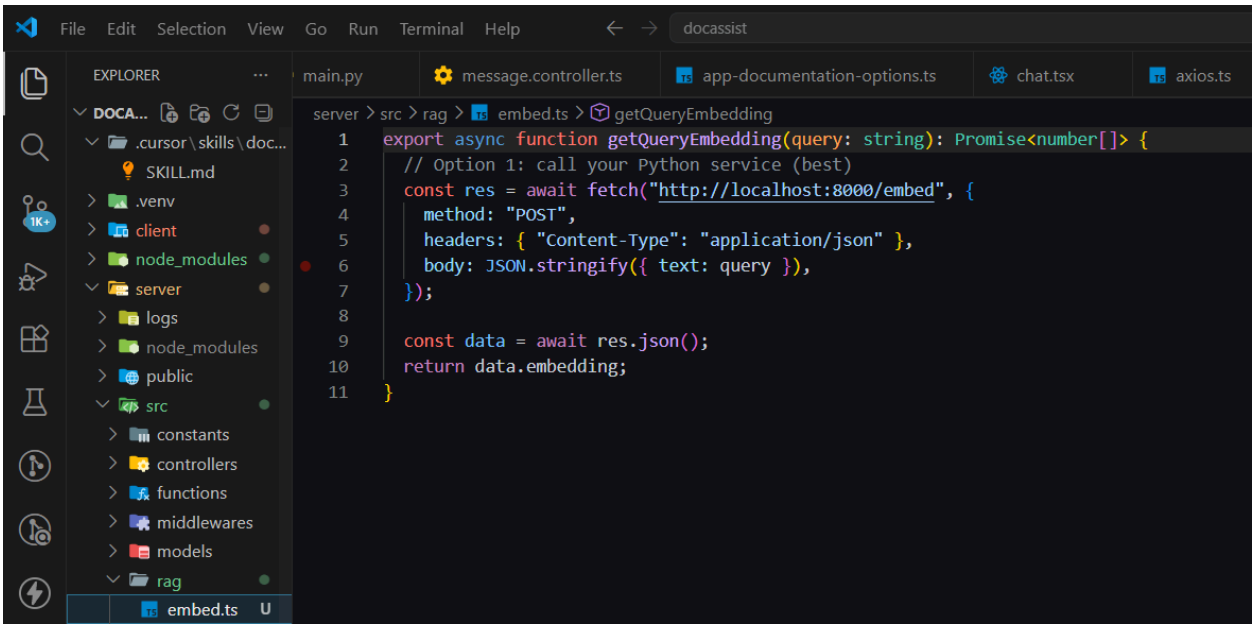

Figure 5.2.1 Query Embedding

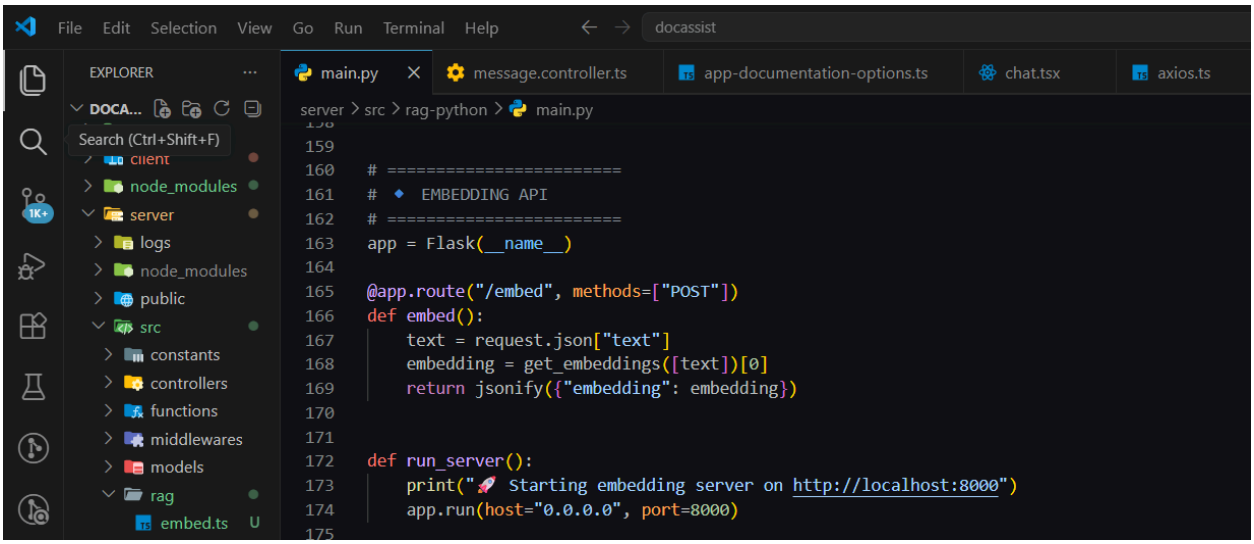

Figure 5.2.2 Documentation Embeddings

5.2.2 Document Retrieval (Vector Search)

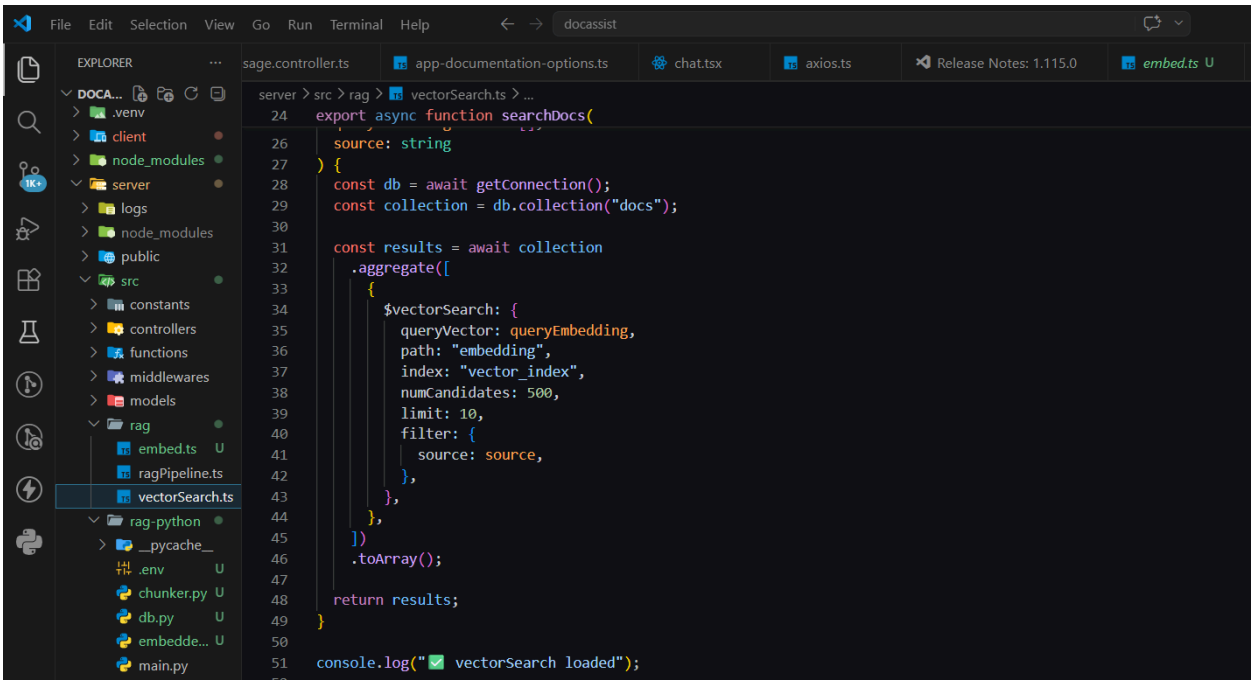

Figure 5.2.3 Retrieving similar vectors from selected source

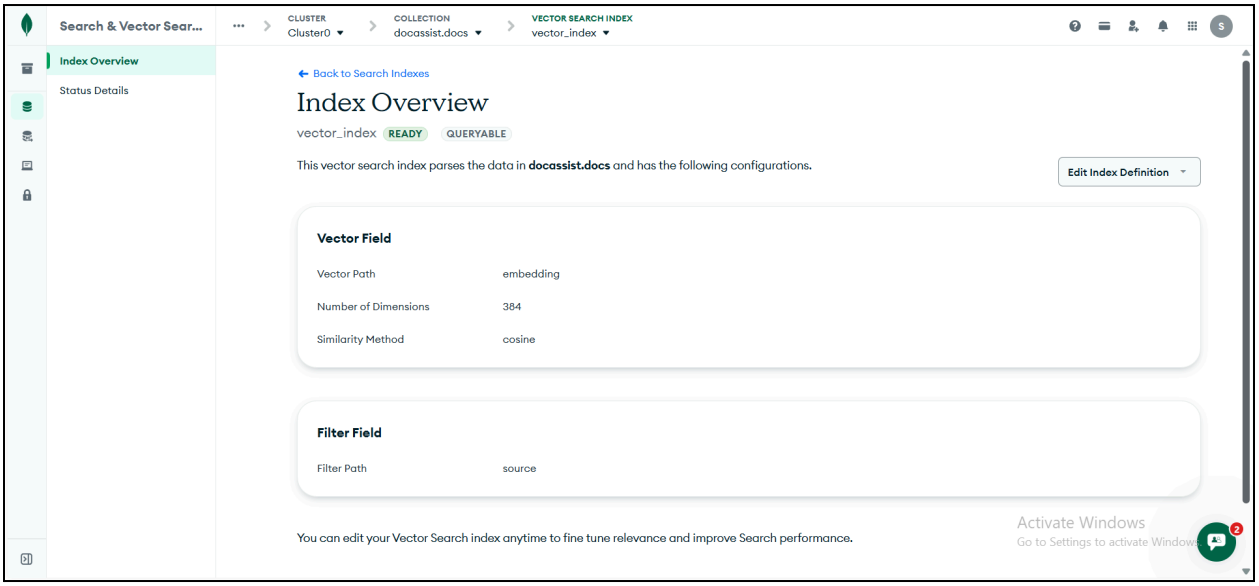

Figure 5.2.4 Vector index configuration

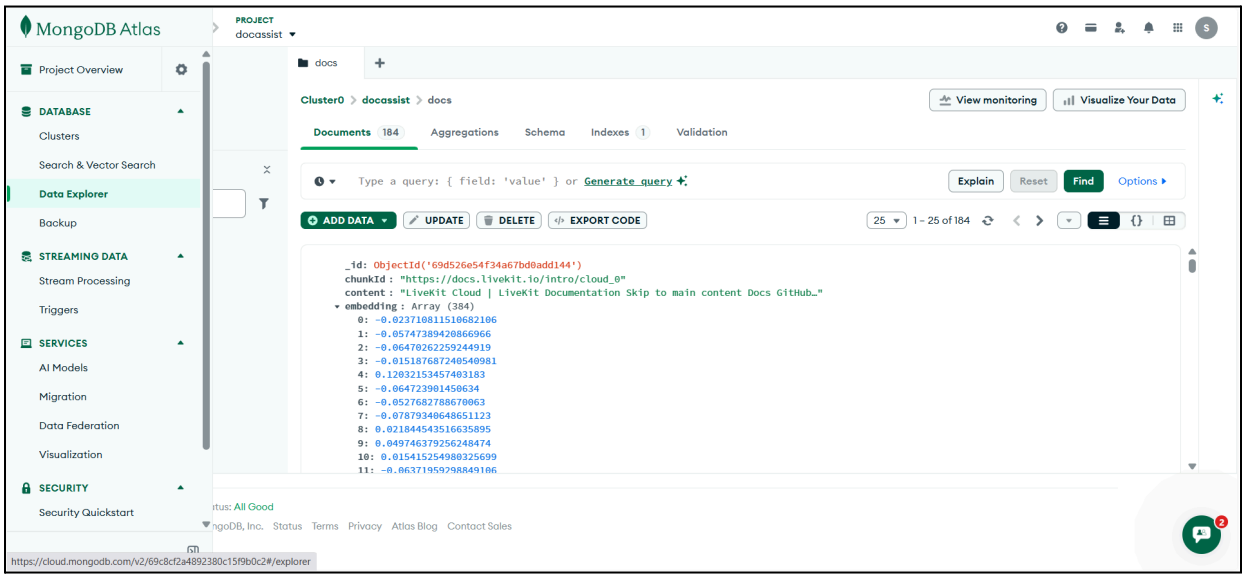

Figure 5.2.5 Stored document with embedding in MongoDB

5.2.3 Response Generation (RAG Pipeline)

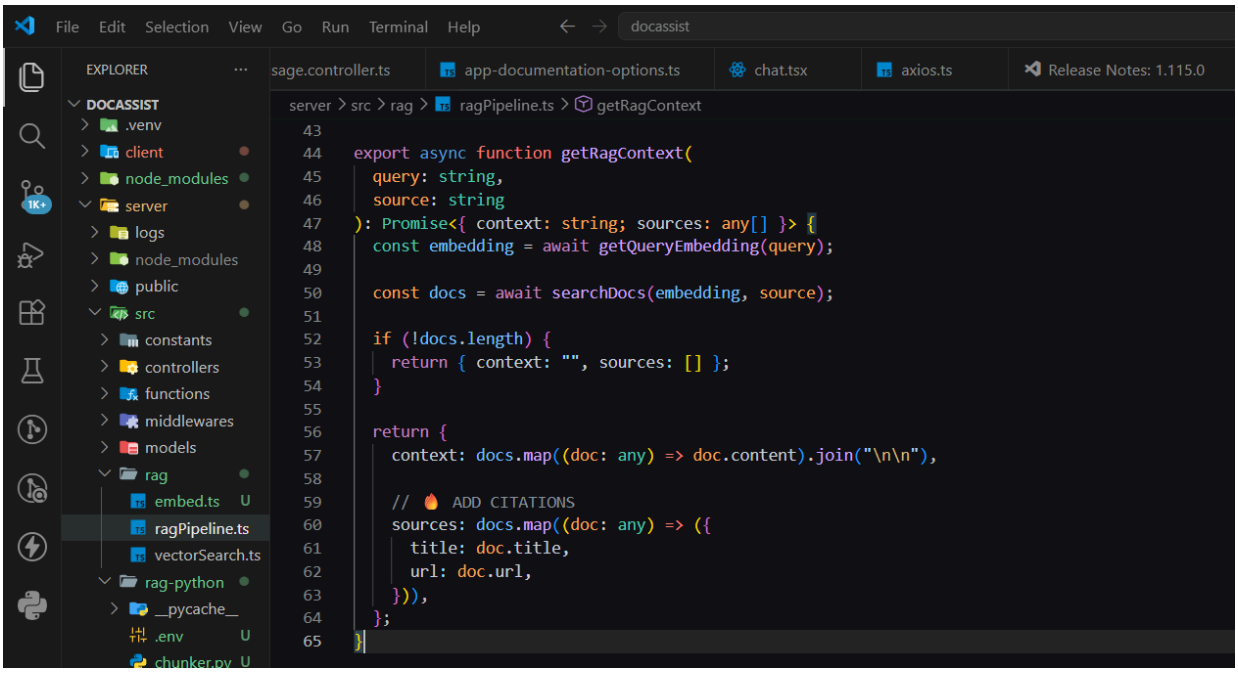

Figure 5.2.6 Get context from selected documentation

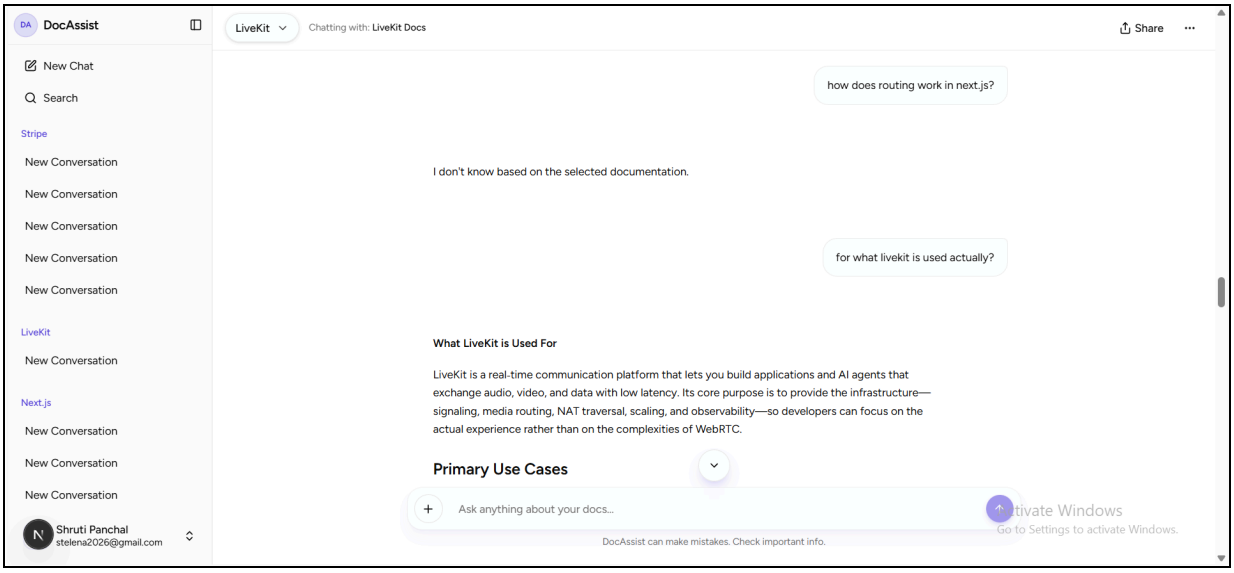

Figure 5.2.7 Isolated documentation response

6. SYSTEM IMPLEMENTATION & TESTING

6.1 CODING STANDARDS

The DocAssist system follows structured coding standards to ensure maintainability, readability, and scalability of the codebase. Standard practices such as modular design, consistent naming conventions, and proper error handling were followed across all components of the system.

6.1.1 Frontend (Next.js & React)

- Used **TypeScript** for type safety and better code reliability
- Followed **component-based architecture** for reusable UI elements
- Maintained clear folder structure (components, pages, hooks)
- Used **Tailwind CSS** for consistent styling
- Ensured proper state management and separation of concerns
- Implemented clean handling of API responses and streaming data

6.1.2 Backend (Node.js + Express.js)

- Followed **modular architecture** (controllers, services, utils)
- Used **TypeScript** for structured backend development
- Implemented proper **error handling and middleware usage**
- Maintained clean API design for communication with frontend
- Used **async/await** for handling asynchronous operations
- Ensured separation between business logic and routing

6.1.3 Database (MongoDB Atlas)

- Used **NoSQL schema design** for flexible data storage
- Stored document chunks along with vector embeddings
- Implemented **vector indexing** for similarity search
- Used proper field naming (content, embedding, source, url)
- Ensured efficient querying using filters and indexing

6.1.4 Version Control

- Used **Git** for version control
- Maintained project repository on **GitHub**
- Followed structured commit messages
- Managed different features through incremental updates
- Ensured code backup and collaboration support

6.2 TESTING METHODS

6.2.1 Environment-based Testing Flow

The system was tested in a local development environment where all components (frontend, backend, and embedding service) were run simultaneously.

The testing flow included:

- Running the **Python embedding server** (Flask)
- Running the **Node.js backend server**
- Running the **Next.js frontend application**
- Verifying end-to-end communication between all components

This ensured that the system worked as an integrated pipeline from user input to response generation.

6.2.2 Manual Testing Activities

Manual testing was performed to validate the functionality and correctness of the system:

- Tested query responses for different documentation sources (LiveKit, Stripe, Next.js)
- Verified **context isolation** (ensuring responses are from selected documentation only)
- Checked system behavior for invalid or unrelated queries
- Validated real-time response streaming using SSE
- Tested citation display along with responses
- Verified user authentication and conversation handling

6.2.3 Limitations

- System performance depends on external APIs (LLM and embedding models)
- Retrieval accuracy may vary depending on quality of scraped data
- Limited to predefined documentation sources
- Response time may increase with large datasets
- No automated testing framework implemented (manual testing used)

7. FUTURE ENHANCEMENT

7.1 FUTURE SCOPE AND ENHANCEMENTS

The DocAssist system successfully demonstrates the implementation of a multi-document Retrieval-Augmented Generation (RAG) pipeline that enables users to query technical documentation using natural language. The system integrates multiple components including frontend interaction, backend processing, embedding generation, vector search, and LLM-based response generation. Additionally, features such as context isolation and citation support enhance the accuracy and reliability of responses.

However, as the system is developed as a prototype, there are several opportunities for further improvement and expansion. Enhancements can be made in terms of system performance, scalability, user experience, and feature capabilities. By incorporating advanced optimization techniques, improving interface design, and extending support for additional functionalities, the system can be transformed into a more robust and production-ready solution. The following sections outline key areas where future improvements can be implemented.

7.1.1 Implementing Automated Test Cases

Currently, the system relies on manual testing to validate functionality. In the future, automated testing frameworks can be integrated to ensure reliability and consistency.

- Integration of unit testing for backend modules
- Automated API testing for embedding and retrieval services
- End-to-end testing for complete user flow
- Use of tools like Jest or Cypress for frontend and backend testing

7.1.2 Performance Optimization and Scalability

As the system grows with more documentation sources and users, performance optimization becomes essential.

- Optimize vector search queries for faster retrieval
- Implement caching mechanisms for frequently asked queries
- Improve embedding generation efficiency
- Scale backend services for handling multiple concurrent users

7.1.3 Enhanced User Interface and Experience

The user interface can be further improved to enhance usability and interaction.

- Improved chat UI with better message formatting
- Highlighting relevant parts of retrieved content
- Better visualization of citations and sources
- Adding search history and saved queries

7.1.4 Expanding Documentation and Feature Support

The system can be extended to support more features and documentation sources.

- Integration of additional documentation sources (Firebase, OpenAI, etc.)
- Support for document upload and custom knowledge bases
- Multi-language query support
- Advanced filtering and search options

8. CONCLUSION

8.1 SELF ANALYSIS OF PROJECT LIABILITIES

The DocAssist system demonstrates strong viability as an AI-powered documentation assistant. The system successfully integrates multiple components, including a chat-based frontend, a Node.js backend, a Python-based embedding service, and MongoDB Atlas for vector storage and retrieval.

From a technical perspective, the system is scalable and modular, allowing easy integration of additional documentation sources and features. The use of a Retrieval-Augmented Generation (RAG) approach ensures that responses are grounded in actual documentation, improving reliability and reducing hallucination.

From a usability standpoint, the system provides a simple and intuitive interface that enables users to interact with complex documentation using natural language queries. Features such as context isolation and citation support further enhance user trust and experience.

Overall, the project is feasible for further development into a production-level system with additional optimization and enhancements.

8.2 PROBLEMS ENCOUNTERED AND THEIR SOLUTIONS

During the development of the DocAssist system, several challenges were encountered across different components. These challenges were addressed through systematic debugging and iterative improvements.

8.2.1 MongoDB Atlas Connection and Authentication Issues

Problem:
Initial attempts to connect to MongoDB Atlas resulted in authentication failures and connection errors.

Solution:
The issue was resolved by correctly configuring the connection URI, encoding special characters in the password, and allowing network access through IP whitelisting in MongoDB Atlas.

8.2.2 Multi-Documentation Retrieval Issues

Problem:
The system initially retrieved responses from incorrect documentation sources, leading to inaccurate answers.

Solution:
This was resolved by implementing source-based filtering in the vector search query and ensuring that the selected documentation is passed throughout the RAG pipeline.

8.2.3 Embedding Service Integration Issues

Problem:
There were issues in communication between the Node.js backend and the Python embedding service, causing request failures.

Solution:
The issue was fixed by ensuring the embedding server runs properly, correcting API endpoints, and handling request-response flow using REST APIs.

8.2.4 SSE Streaming and Data Transfer Issues

Problem:
Citations (sources) were not reaching the front end due to improper handling of SSE responses.

Solution:
The SSE parsing logic was updated to include additional fields such as sources, and frontend handlers were modified to correctly process and display this data.

8.2.5 Manual Testing Effort

Problem:
Testing was performed manually, which required significant time and effort to validate different scenarios.

Solution:
A structured testing approach was followed, including testing for different documentation sources, edge cases, and system behaviors to ensure reliability.

8.3 SUMMARY OF PROJECT WORK

8.3.1 Implementation of DocAssist System

The project successfully developed a documentation-based AI assistant that allows users to query multiple documentation sources using natural language. The system integrates web scraping, text chunking, embedding generation, vector storage, and LLM-based response generation into a unified pipeline.

Key features implemented include:

- Multi-documentation support (LiveKit, Stripe, Next.js)

- Context-aware retrieval using vector search
- Real-time response streaming using SSE
- Citation support for improved transparency
- Conversation-based interaction

8.3.2 RAG-Based System Development

A complete Retrieval-Augmented Generation pipeline was implemented, enabling the system to retrieve relevant document chunks and generate accurate responses based on context.

The system ensures:

- Reduced hallucination by restricting responses to retrieved context
- Improved answer accuracy through semantic search
- Scalable architecture for adding more documentation sources

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[3] MongoDB, "MongoDB Atlas Documentation," MongoDB Inc., Available: https://www.mongodb.com/docs/atlas/

[4] Next.js, "Next.js Documentation," Vercel Inc., Available: https://nextjs.org/docs

[5] Node.js, "Node.js Documentation," OpenJS Foundation, Available: https://nodejs.org/en/docs

[6] Flask, "Flask Documentation," Pallets Projects, Available: https://flask.palletsprojects.com/

[7] Hugging Face, "Transformers Documentation," Hugging Face Inc., Available: https://huggingface.co/docs/transformers

[8] Groq, "Groq API Documentation," Groq Inc., Available: https://console.groq.com/docs

[9] Stripe, "Stripe Documentation," Stripe Inc., Available: https://docs.stripe.com/

[10] LiveKit, "LiveKit Documentation," LiveKit Inc., Available: https://docs.livekit.io/

[11] MDN Web Docs, "Server-Sent Events," Mozilla Developer Network, Available: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events

Shruti PANCHAL

DocAssist – AI-Powered Documentation Assistant with RAG Architecture

Sem-8 Major Project Report
22CS Batch
Charotar University of Science And Technology

Document Details

Submission ID
trn:oid:::1:3536913114

Submission Date
Apr 14, 2026, 10:16 AM GMT+5:30

Download Date
Apr 14, 2026, 10:37 AM GMT+5:30

File Name
SGP_SEM_8_Report_Shruti.pdf

File Size
9.7 MB

32 Pages

4,669 Words

36,386 Characters

2% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

8 Not Cited or Quoted 2%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
0% Submitted works (Student Papers)

Match Groups

- **8** Not Cited or Quoted 2%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Internet

www.slideshare.net **2%**